QUANTUMSENSE: AN ADAPTIVE ML-KEM SCHEME SELECTION FRAMEWORK FOR LATENCY-CONSTRAINED SATELLITE TELEMETRY PIPELINES

Yuxing Yang*, Shuhe Cui**, Kevin Ziru Wei†

*School of Information Sciences, University of Illinois Urbana-Champaign, Champaign, IL

**College of Design and Engineering, National University of Singapore, Singapore

†Information School, University of Washington, Seattle, WA

E-mails: yuxingy2@illinois.edu, E1718781@u.nus.edu, kzw6@uw.edu

ABSTRACT

Post-quantum migration in satellite telemetry introduces a core tension: post-quantum and hybrid handshakes can increase connection-setup cost, while bandwidth-constrained telemetry links impose strict latency budgets. Fixed-scheme deployments are simple to audit, but they cannot react when round-trip time, available bandwidth, or concurrent handshake load changes during operation. We propose QuantumSense, a lightweight adaptive framework that evaluates seven key-exchange configurations: elliptic-curve Diffie–Hellman (ECDH), the Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM) at three security categories, and three corresponding ECDH–ML-KEM hybrid configurations. During online replay, QuantumSense selects only from an operator-approved action set containing an ML-KEM component. The framework uses a contextual bandit policy based on round-trip time (RTT), available bandwidth, and concurrent handshake load. Using Ground Station–Test Article (GS–TA) testbed measurements, QuantumSense first selects Hybrid-768 within 15 handshakes after a simulated regime change. When measurements from the evaluated high-concurrency settings are pooled, QuantumSense reduces mean latency by 27.3% and 99th-percentile (P99) latency by 17.3% relative to static Hybrid-512, while classical-only ECDH remains outside the online action set.

INTRODUCTION

Satellite telemetry systems need secure key establishment before mission data can be protected and streamed. A delayed handshake delays the point where the telemetry pipeline can begin secure data transfer, so handshake latency is part of the operational cost of the system, not only a setup detail. This issue becomes more important as telemetry systems move toward post-quantum cryptography. Classical public key methods such as elliptic-curve Diffie–Hellman are efficient, but they do not provide long-term protection against a future quantum computer [1]. NIST has standardized ML-KEM at three security levels [2]. Hybrid key exchange is also used as a transition approach that combines classical and post-quantum key establishment [3]. The CRYSTALS-Kyber work behind ML-KEM also showed that lattice-based key encapsulation can be practical for real systems [4].

Prior work has measured post-quantum key exchange in TLS and shown that its overhead depends on both protocol choice and network conditions [5, 6]. Recent ITC work has also shown that post-quantum key encapsulation is relevant to aeronautical mobile telemetry, especially for distributing symmetric keys between telemetry parties [7].

A Ground Station to Test Article link can support several key exchange choices. These include classical ECDH, ML-KEM at multiple security levels, and hybrid combinations that pair a classical exchange with a post-quantum encapsulation. These choices differ in payload size, compute cost, and latency. In this paper, all seven configurations are measured in the testbed, but classical ECDH is used only as a baseline during online operation. The selector chooses only from an operator-approved post-quantum or hybrid action set. This distinction is important because a learning policy should not be allowed to lower the security boundary in order to reduce latency.

Static scheme selection is simple and easy to audit, but telemetry links rarely stay in one condition. round-trip time can change with link geometry, bandwidth can change with link quality and ground station conditions, and connection load can rise when many secure sessions start together. A scheme chosen for one condition may add avoidable latency in another. Hybrid schemes are especially sensitive to this issue because they add both classical and post-quantum work to each handshake. This paper does not propose a new cryptographic algorithm. Instead, it studies how to select among approved post-quantum and hybrid schemes as link conditions change.

This paper presents QuantumSense, an adaptive framework for selecting post-quantum key exchange schemes in latency-constrained telemetry pipelines. QuantumSense observes round-trip time, available bandwidth, and concurrent connection count before each handshake. It then selects one allowed scheme for that handshake using a contextual bandit policy that updates its latency estimates from measured handshakes. This paper makes three contributions. First, it formulates key exchange scheme selection for telemetry links as a contextual bandit problem with a fixed operator-approved action set. Second, it implements this idea with a disjoint LinUCB policy that uses round-trip time, bandwidth, and concurrency as context. Third, it evaluates the policy with measured latency data from a Ground Station to Test Article testbed and compares it with static classical, post-quantum, and hybrid baselines.

BACKGROUND AND RELATED WORK

A. Post-Quantum Key Establishment in Telemetry

Telemetry systems usually protect mission data with symmetric encryption after a session key has been established. The key exchange step therefore sits at the start of the secure telemetry pipeline. Classical public key methods such as RSA and elliptic-curve Diffie–Hellman are widely used today, but their long-term security is threatened by quantum algorithms. Shor's algorithm can solve integer factoring and discrete logarithms in polynomial time on a sufficiently capable quantum computer [1]. Symmetric cryptography faces a smaller quantum impact: Grover's algorithm gives a quadratic speedup, which is commonly addressed by increasing symmetric key length [8].

Post-quantum migration therefore focuses heavily on key establishment. NIST standardized ML-KEM from the CRYSTALS-Kyber family at three security levels [2, 4]. Earlier work such as NewHope and ring-LWE key exchange for TLS showed that lattice-based key exchange could be implemented in real network protocols [9, 5]. Telemetry adds its own constraints because key establishment must fit within mission timing, link bandwidth, and ground station processing limits. Recent ITC work has studied post-quantum key encapsulation for aeronautical mobile telemetry, including the use of code-based KEMs to distribute symmetric keys between telemetry parties [7]. These results show that post-quantum key establishment is not only a general Internet security issue; it is also a telemetry system design issue.

B. PQC and Hybrid Key Exchange in TLS

Hybrid key exchange is a common transition approach during post-quantum migration. A hybrid design combines a classical exchange with a post-quantum encapsulation so that the session does not rely on only one new primitive during migration. Formal work on hybrid key encapsulation and authenticated key exchange gives a foundation for this approach [3]. In deployed protocol settings, TLS 1.3 provides the handshake structure used by many post-quantum key exchange experiments [10]. The benefit of a hybrid scheme is migration conservatism. The cost is that each handshake carries more data and does more cryptographic work.

Prior measurements show that this overhead is not constant. Paquin, Stebila, and Tamvada benchmarked post-quantum cryptography in TLS and showed that the observed cost depends on the KEM, the protocol setting, and the network environment [6]. Sikeridis, Kampanakis, and Devetsikiotis reported similar performance variation for post-quantum authentication in TLS 1.3 [11]. The Open Quantum Safe project also provides the software foundation used by many experimental post-quantum protocol studies [12]. These works support the main motivation of this paper: a fixed key exchange choice may be easy to configure, but it can leave performance headroom unused when link conditions and server load change.

C. Contextual Bandits for Online Decision-Making

A contextual bandit models sequential decisions under partial feedback. At each step, the policy observes a context, selects one action, and then observes the outcome of only that selected action. This is different from a full supervised learning setting, where labels for all possible actions would be available. Key exchange selection has the same feedback pattern. The telemetry system observes the latency of the scheme it used for a handshake, but it does not observe the latency that the other schemes would have produced for that same handshake.

The Upper Confidence Bound idea is a standard way to balance exploration and exploitation in bandit problems [13]. LinUCB extends this idea to contextual decisions by fitting a linear reward model and adding an uncertainty bonus to each action score [14]. Later work developed regret bounds for linear stochastic bandits [15]. Telemetry links can also be non-stationary: round-trip time, bandwidth, and connection load may change across a pass or during a burst of connections. Work on switching bandit problems shows why fixed exploration can be slow after a reward distribution changes [16]. QuantumSense applies these ideas to telemetry key exchange selection. The

context is the current link state, the actions are the allowed post-quantum and hybrid schemes, and the reward is negative handshake latency.

ADAPTIVE SCHEME SELECTION PROBLEM

A. *Telemetry Link State*

We consider a Ground Station, denoted as GS, that establishes secure sessions with one or more Test Articles, denoted as TA. Each session starts with a key exchange handshake. The handshake creates a shared secret, and the telemetry system uses this secret to derive symmetric keys for mission data. Since mission data cannot be protected by the new session until the handshake completes, the key exchange latency directly affects the start of secure telemetry flow.

Before each handshake, the GS observes three link variables. The first variable is round-trip time, denoted by Δ. The second variable is available bandwidth, denoted by W. The third variable is concurrent handshake load, denoted by n. These variables form the context at handshake t:

$$x_t = (\Delta_t, W_t, n_t). \tag{1}$$

round-trip time captures propagation and path delay. Bandwidth captures the cost of larger handshake messages. Concurrent load captures server pressure when many secure sessions start together. These three variables are not a complete model of a telemetry link, but they capture the main latency drivers used in this study.

B. *Measured Schemes and Online Action Set*

The testbed measures seven key exchange configurations:

$$\mathcal{A}_{\text{measured}} = \{\text{ECDH}\} \cup \{\text{ML-KEM}_k, \text{Hybrid}_k \mid k \in \{512, 768, 1024\}\}. \tag{2}$$

This measured set includes one classical baseline, three ML-KEM configurations, and three hybrid configurations. Classical ECDH is useful for comparison because it shows the latency of a conventional pre-quantum deployment. It is not treated as an allowed online choice.

The online selector chooses only from the operator-approved post-quantum and hybrid action set:

$$\mathcal{A}_{\text{online}} = \mathcal{A}_{\text{measured}} \setminus \{\text{ECDH}\}. \tag{3}$$

This separation fixes the security boundary before learning starts. The testbed can measure ECDH, but the online policy cannot select it. A mission operator can also define a stricter online set if a higher minimum security level is required.

Table 1: Measured key exchange configurations and online availability.

Configuration	Type	Security role	Online choice
ECDH	Classical	Baseline only	No
ML-KEM-512	Post-quantum	NIST Category 1	Yes
ML-KEM-768	Post-quantum	NIST Category 3	Yes
ML-KEM-1024	Post-quantum	NIST Category 5	Yes
Hybrid-512	Hybrid	ECDH plus ML-KEM-512	Yes
Hybrid-768	Hybrid	ECDH plus ML-KEM-768	Yes
Hybrid-1024	Hybrid	ECDH plus ML-KEM-1024	Yes

C. *Latency Objective*

At handshake t, the policy observes x_t and selects one scheme $a_t \in \mathcal{A}_{\text{online}}$. After the handshake ends, the system observes the latency of the selected scheme, denoted by $\ell_t(a_t)$. The reward is defined as negative latency:

$$r_t = -\ell_t(a_t). \tag{4}$$

This reward makes lower handshake latency correspond to higher reward.

The objective is to reduce total expected handshake latency over T handshakes:

$$\min_{\{a_t\}} \mathbb{E}\left[\sum_{t=1}^{T} \ell_t(a_t)\right], \tag{5}$$

subject to

$$a_t \in \mathcal{A}_{\text{online}} \quad \text{for all } t. \tag{6}$$

The constraint in Equation 6 is structural. It does not depend on the learned latency estimates and does not change with the network state.

The system observes only the latency of the scheme it selects. It does not observe the latency of every other scheme for the same handshake. This partial feedback makes the problem a contextual bandit problem. The policy optimizes per-handshake latency. Mean latency and P99 latency are reported later as evaluation metrics, with P99 used to show tail behavior under load.

QUANTUMSENSE DESIGN

A. *System Flow*

QuantumSense has two connected parts. The first part is the GS to TA measurement path, which runs TLS 1.3 handshakes for each measured key exchange configuration and records the resulting latency under controlled link conditions. The second part is the online selector, which uses the current telemetry link state to select one scheme from $\mathcal{A}_{\text{online}}$ before each handshake. The selector is not allowed to select classical ECDH, because ECDH is outside the online action set defined in Section III.

At each handshake, the selector receives the current context $x_t = (\Delta_t, W_t, n_t)$, scores each allowed scheme, and selects the scheme with the highest score. After the handshake completes, the system observes the measured latency of the selected scheme and updates only that scheme model. The selector does not measure the schemes that were not selected at that step. This matches live telemetry operation, where running extra handshakes only to test unused schemes would add delay and traffic. The contextual bandit structure is used because it learns from this partial feedback while still allowing controlled exploration among approved schemes.

B. Context Features and LinUCB Selection

QuantumSense maps the raw telemetry context into a feature vector before scoring the candidate schemes. For a context $x_t = (\Delta_t, W_t, n_t)$, the feature vector is

$$\phi(\Delta, W, n) = \begin{bmatrix} \Delta \\ \log(W+1) \\ \log(n+1) \\ \Delta \log(W+1) \\ \Delta \log(n+1) \\ \log(W+1)\log(n+1) \end{bmatrix}. \tag{7}$$

The logarithmic terms reduce the scale gap between narrow and wide bandwidth values, and between light and heavy connection load. The interaction terms allow the model to capture simple combined effects. For example, bandwidth can matter more when propagation delay is low, and connection load can matter more when the server is already busy. The feature dimension is $d = 6$.

QuantumSense uses a disjoint LinUCB policy [14]. The policy keeps a separate linear reward model for each allowed scheme. For each scheme $a \in \mathcal{A}_{\text{online}}$, the policy stores a design matrix $A_a \in \mathbb{R}^{d \times d}$ and a response vector $b_a \in \mathbb{R}^d$. The matrix is initialized as the identity matrix, and the response vector is initialized as zero. The estimated parameter vector for scheme a is

$$\hat{\theta}_a = A_a^{-1} b_a. \tag{8}$$

At handshake t, the policy computes $\phi_t = \phi(x_t)$ and assigns each allowed scheme an upper-confidence score:

$$s_t(a) = \hat{\theta}_a^\top \phi_t + \alpha_t \sqrt{\phi_t^\top A_a^{-1} \phi_t}. \tag{9}$$

The first term is the estimated reward, which corresponds to negative handshake latency. The second term is the uncertainty bonus. This bonus follows the upper-confidence-bound principle used in bandit learning [13, 14]. A larger value of α_t increases exploration, while a smaller value makes the policy rely more on the current latency estimates.

The selected scheme is

$$a_t = \arg \max_{a \in \mathcal{A}_{\text{online}}} s_t(a). \tag{10}$$

The maximization is only over $\mathcal{A}_{\text{online}}$, so the policy cannot select ECDH. After the handshake finishes, the policy observes $r_t = -\ell_t(a_t)$ and updates the model for the selected scheme:

$$A_{a_t} \leftarrow A_{a_t} + \phi_t \phi_t^\top, \tag{11}$$

$$b_{a_t} \leftarrow b_{a_t} + r_t \phi_t. \tag{12}$$

The models for schemes that were not selected are unchanged at that round. This is the key difference between the online selector and a full supervised model trained with labels for every possible scheme.

C. *Handling Link Changes*

Telemetry links can change during operation. A policy that has learned a good choice under one link state may perform poorly after a shift in RTT, bandwidth, or connection load. QuantumSense therefore monitors recent rewards and temporarily increases exploration when the reward pattern changes. This mechanism is not used to change the security policy. It only changes how aggressively the selector explores schemes that are already inside $\mathcal{A}_{\text{online}}$.

The detector uses two sliding windows of length m. The recent window contains the latest m rewards, and the previous window contains the m rewards before that. Let $\bar{r}_{\text{new}}$ be the mean reward in the recent window, let $\bar{r}_{\text{old}}$ be the mean reward in the previous window, and let σ_{old} be the standard deviation of the previous window. The change score is

$$z_t = \frac{|\bar{r}_{\text{new}} - \bar{r}_{\text{old}}|}{\sigma_{\text{old}} + \epsilon}. \tag{13}$$

The small constant ϵ prevents division by zero. If z_t is larger than a threshold τ, QuantumSense treats the event as a possible link change and temporarily increases the exploration weight:

$$\alpha_t = \gamma \alpha_0, \quad \gamma > 1. \tag{14}$$

Here, α_0 is the base exploration weight and γ controls the size of the exploration boost. After the boost period ends, the policy returns to α_0. This lets the selector re-check allowed schemes after a link change without ever leaving the operator-approved action set.

TESTBED AND EVALUATION METHOD

A. *GS to TA Testbed*

We evaluate QuantumSense with a Ground Station to Test Article testbed. The testbed runs TLS 1.3 handshakes for each measured key exchange configuration [10]. All configurations use the same application path, transport path, and measurement procedure. This keeps the comparison focused on the key exchange choice rather than on differences in the surrounding software stack or network path.

The cryptographic stack uses OpenSSL with Open Quantum Safe support. Open Quantum Safe is

used because it provides implementations and protocol integrations for post-quantum key exchange experiments [12]. The measured configurations are classical ECDH, ML-KEM-512, ML-KEM-768, ML-KEM-1024, Hybrid-512, Hybrid-768, and Hybrid-1024. The ML-KEM configurations follow the NIST standardized ML-KEM family [2]. Classical ECDH is measured only as a baseline and is not included in $\mathcal{A}_{\text{online}}$.

The testbed records TLS handshake completion time. A measurement starts when the client begins the handshake and ends when the handshake completes and the secure session is ready for application data. This measurement captures the cost that matters to the telemetry pipeline: mission data cannot be protected by the new session key until the handshake finishes.

Table 2: Summary of the experimental testbed.

Item	Configuration
Protocol	TLS 1.3
Crypto support	OpenSSL with Open Quantum Safe support
Measured schemes	ECDH, ML-KEM, and Hybrid key exchange
Link variables	RTT, bandwidth, and concurrent handshakes
Latency metric	TLS handshake completion time
Classical ECDH role	Baseline only
Online action set	ML-KEM and Hybrid configurations

B. Network and Load Conditions

The evaluation controls three link variables. The first variable is round trip time Δ. The second variable is available bandwidth W. The third variable is concurrent connection count n. These variables match the context used by the online selector and are chosen because they directly affect handshake latency in a telemetry setting. RTT captures path delay, bandwidth captures the cost of larger handshake messages, and concurrency captures ground station load during bursts of secure session establishment.

The measurement campaign has three parts. First, each scheme is measured with no added impairment to establish a local baseline. Second, network sensitivity is measured across RTT and bandwidth settings:

$$\Delta \in \{0, 10, 50, 200\} \text{ ms}, \tag{15}$$

$$W \in \{1, 10, W_{\text{uncapped}}\} \text{ Mbps}. \tag{16}$$

Here, W_{uncapped} denotes the measured throughput of the testbed path when no bandwidth cap is applied. This avoids treating the uncapped case as an infinite value in the feature vector.

Third, concurrency sensitivity is measured by running simultaneous client handshakes with

$$n \in \{1, 10, 50, 100, 200\}. \tag{17}$$

A transparent network control layer adds delay and bandwidth limits for the network sensitivity tests. For the concurrency tests, clients start through a synchronization barrier so that the server receives a burst of handshake requests. The same procedure is used for every measured scheme.

C. Metrics and Baselines

Each measurement produces one labeled observation containing the context, the key exchange configuration, and the measured handshake latency. These observations are used in two ways. First, they provide direct static baseline results. Second, they provide latency feedback for the online replay evaluation. In the replay evaluation, the policy observes the current context, selects one scheme from $\mathcal{A}_{\text{online}}$, receives the measured latency for that selected scheme, and updates only the selected scheme model. It does not receive the latency of schemes it did not select at that step.

We report mean latency and P99 latency. Mean latency measures the typical handshake completion time. P99 latency measures tail behavior under load. P99 is important for telemetry because a small number of slow handshakes can delay downstream secure data flow. The bandit reward uses per-handshake latency, not P99 latency directly. P99 is reported as an evaluation metric to show whether the selected schemes also reduce tail latency.

For each comparison, latency reduction relative to a static baseline is computed as

$$\text{Reduction} = \frac{L_{\text{baseline}} - L_{\text{QuantumSense}}}{L_{\text{baseline}}} \times 100\%. \tag{18}$$

Here, L is either mean latency or P99 latency. A positive value means that QuantumSense has lower latency than the static baseline.

The static baselines use one key exchange configuration for every handshake and do not adapt to RTT, bandwidth, or connection load. They include all seven measured configurations. Hybrid-512 is the main comparison point because it represents a common migration choice that combines a classical exchange with a post-quantum encapsulation. Classical ECDH is included only to show the latency of a classical deployment and is not treated as an allowed online choice.

EVALUATION RESULTS

A. High Concurrency Performance

We first evaluate QuantumSense under high concurrency, because this is where handshake cost is most visible to the ground station. During a busy telemetry period, many secure sessions may start close together. Hybrid schemes perform both a classical exchange and an ML-KEM encapsulation, so their added work can increase server-side delay under load. Table 3 compares QuantumSense with static Hybrid-512 at $n = 50$, $n = 100$, and $n = 200$ concurrent handshakes.

Table 3: Mean latency of QuantumSense and static Hybrid-512 under high concurrency.

n	QS Choice	QS Mean (ms)	Hybrid-512 Mean (ms)	Reduction
50	ML-KEM-768	132.3	156.9	15.7%
100	ML-KEM-768	215.2	324.7	33.7%
200	ML-KEM-768	558.2	764.0	26.9%

QuantumSense selects ML-KEM-768 in all three high-concurrency cases. The mean latency reduction relative to static Hybrid-512 is 15.7% at $n = 50$, 33.7% at $n = 100$, and 26.9% at $n = 200$. When latency values are pooled across the three concurrency levels, QuantumSense reduces mean latency by 27.3% relative to static Hybrid-512. This result shows that avoiding hybrid overhead can be useful when the GS is under concurrent handshake load.

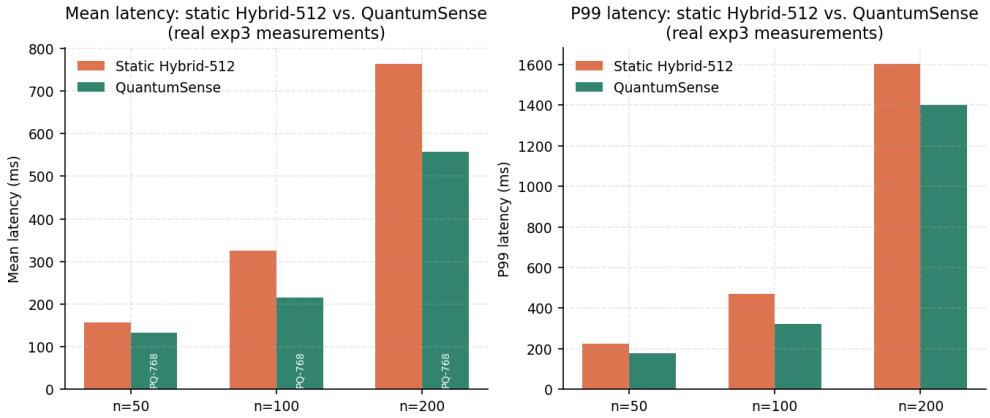


Figure 1: Mean and P99 latency for static Hybrid-512 and QuantumSense at $n = 50$, $n = 100$, and $n = 200$ concurrent clients.

Figure 1 reports both mean latency and P99 latency. The P99 result matters because a small number of slow handshakes can delay the start of protected telemetry flow. Across the same high-concurrency cases, QuantumSense reduces P99 latency by 17.3% relative to static Hybrid-512.

B. Static Baseline Comparison

We next compare QuantumSense with static scheme choices. A static scheme uses the same key exchange configuration for every handshake and does not adapt to RTT, bandwidth, or connection load. Table 4 reports the reduction achieved by QuantumSense relative to each static baseline, computed from measurements pooled over $n = 50$, $n = 100$, and $n = 200$.

Table 4: QuantumSense improvement relative to static schemes, computed from measurements pooled over $n = 50$, $n = 100$, and $n = 200$.

Static Scheme	Mean Reduction	P99 Reduction
ECDH	19.6%	9.6%
ML-KEM-512	8.7%	0.7%
ML-KEM-768	0.0%	0.0%
ML-KEM-1024	8.3%	4.2%
Hybrid-512	27.3%	17.3%
Hybrid-768	14.8%	5.3%
Hybrid-1024	17.2%	7.6%

The largest gains appear against the static hybrid baselines. This is expected in the high-concurrency setting because hybrid schemes add extra cryptographic work to every handshake. QuantumSense

matches static ML-KEM-768 in this subset because the learned policy selects ML-KEM-768 for these high-load cases. This should not be interpreted as a claim that ML-KEM-768 is always the best scheme. It shows that the best choice depends on the operating state. The ECDH row is included only as a performance reference; ECDH is not part of the online action set.

C. Adaptation After Link Degradation

We also evaluate how quickly QuantumSense reacts after a clear link change. The trace has two regimes. The first 100 handshakes use a LEO-style condition with $\Delta = 10$ ms and $W = 10$ Mbps. The next 100 handshakes use a degraded GEO-style condition with $\Delta = 200$ ms and $W = 1$ Mbps. This test is designed to show whether the selector can move away from a previously strong choice when the link state changes.

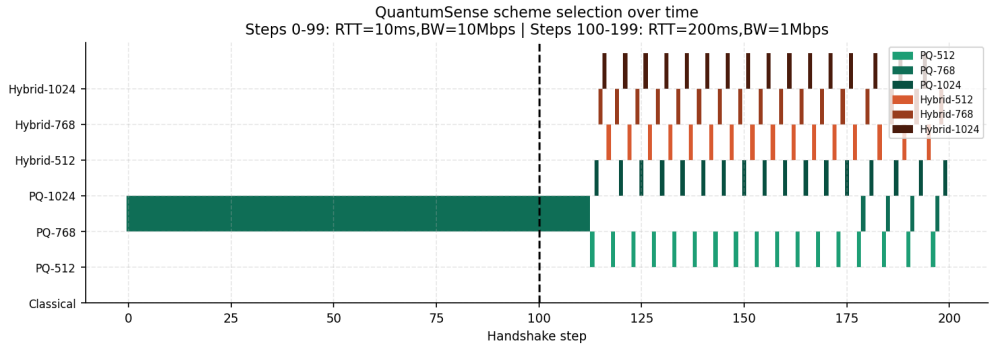


Figure 2: QuantumSense scheme selection before and after a link change at handshake 100.

Figure 2 shows the selected scheme over time. The policy mainly selects ML-KEM-768 before the change. After the link degrades, the change detector increases exploration, and the policy first selects Hybrid-768 within 15 handshakes. Because the selector continues to explore other approved configurations, this value represents time to first post-change selection rather than time to permanent convergence. During this process, ECDH is never selected because it is outside $\mathcal{A}_{\text{online}}$. This result shows that the adaptation mechanism changes the performance choice without changing the security boundary.

D. Representative Telemetry Cases

Table 5 summarizes the policy behavior across representative telemetry operating points. These cases cover low and high RTT, narrow and wide bandwidth, and light and heavy connection load. They are used to illustrate the learned selection behavior across mission-like scenarios, rather than to enumerate every raw measurement condition.

Table 5: QuantumSense selections across representative telemetry cases.

Case	Δ (ms)	W (Mbps)	n	QS Choice	Gain vs. Hybrid-512
LEO normal	10	10	1	Hybrid-512	0.0 ms
LEO narrow	10	1	1	Hybrid-512	0.0 ms
LEO concurrent	10	10	100	ML-KEM-768	96.0 ms
GEO normal	200	100	1	Hybrid-768	77.2 ms
GEO concurrent	200	100	100	ML-KEM-768	146.7 ms
Degraded link	200	1	1	ML-KEM-512	7.6 ms
Ground burst	5	100	200	ML-KEM-768	96.7 ms
Emergency	100	1	10	ML-KEM-768	51.1 ms

The representative cases show a simple operating pattern. At low concurrency, QuantumSense often keeps a hybrid scheme because the added hybrid cost is small. At high concurrency, the policy often moves toward an ML-KEM-only scheme because server load makes the extra hybrid work more expensive. The largest gain appears in the GEO concurrent case, where QuantumSense reduces latency by 146.7 ms relative to static Hybrid-512.

DISCUSSION

A. Security Boundary

QuantumSense changes the key exchange choice, but it does not change the security policy. The operator defines $\mathcal{A}_{\text{online}}$ before learning starts, and the selector can only choose from that set. Classical ECDH is measured as a latency baseline, but it is not available during online operation. This separation is important because the learning policy should not be able to lower the security boundary in order to reduce handshake latency.

The same design also supports stricter mission rules. For example, an operator can remove ML-KEM-512 from the online action set, or require hybrid schemes during selected mission phases. QuantumSense would then optimize latency only among the remaining allowed schemes. This makes the selector a policy layer above approved cryptographic mechanisms, not a replacement for the mission security rule. Hybrid key exchange remains a useful migration option because it combines classical and post-quantum key establishment during the transition period [3].

The selector also assumes that the final negotiated key exchange scheme is protected by the TLS handshake transcript [10]. This matters because an attacker should not be able to modify the selected scheme without detection. QuantumSense therefore relies on the existing protocol authentication and transcript checks provided by TLS. It does not change the TLS security model; it only chooses which approved key exchange configuration is offered or selected under the operator policy.

B. *Deployment Lessons and Limitations*

The results suggest two deployment lessons for telemetry systems. First, hybrid key exchange can be reasonable when concurrency is low, because the added hybrid work has limited effect on total handshake latency in that setting. Second, high concurrency can make hybrid overhead visible at the ground station. When many secure sessions start together, an ML-KEM-only scheme can reduce latency while still remaining inside the post-quantum action set. This is the main reason QuantumSense gains the most over static Hybrid-512 in the high-concurrency tests.

The results also show why scheme selection should not be treated as a single fixed deployment choice. RTT, bandwidth, and concurrent load can change during a mission, and the best latency choice can change with them. A static configuration is easier to audit, but it cannot respond to these operating changes. QuantumSense keeps the audit boundary fixed by limiting the selector to $\mathcal{A}_{\text{online}}$, while still allowing the performance choice to adapt inside that boundary.

The current evaluation has limits. The measurements come from a software testbed, which makes the comparison repeatable but may not capture the exact cost profile of flight hardware or embedded processors. The link conditions are also controlled. Real telemetry links may include packet loss, jitter, gradual bandwidth drift, contact window effects, and other mission-specific constraints. These effects may change both latency and adaptation speed.

The link change experiment uses a sharp transition between two known conditions. This is useful for showing that the detector can respond to a clear regime change, but it does not fully model gradual link drift during an orbital pass. The reward function also uses per-handshake latency. P99 latency is reported as an evaluation metric, but it is not directly optimized by the policy. A future version could add a tail-aware reward if the mission objective places stronger emphasis on high-percentile latency.

CONCLUSION

This paper presented QuantumSense, an adaptive scheme selection framework for post-quantum key exchange in satellite telemetry pipelines. The framework observes round-trip time, available bandwidth, and concurrent handshake load before each TLS handshake. It then selects one allowed ML-KEM or hybrid configuration from an operator-approved action set. The goal is to reduce handshake latency without allowing the learning policy to choose classical-only ECDH during online operation.

QuantumSense uses a disjoint LinUCB policy to learn from measured handshake latency under different link and load conditions. The selector updates only the model for the scheme it actually uses, which matches the partial feedback available in live operation. A link-change detector temporarily increases exploration when the recent reward pattern shifts, allowing the policy to adapt after a change in operating state while staying inside the approved action set.

The evaluation shows that adaptive scheme selection is most useful under high concurrency. Across

the evaluated high-concurrency cases, QuantumSense reduces mean latency by 27.3% and P99 latency by 17.3% relative to static Hybrid-512. In the link degradation experiment, the selector first selects a post-change configuration within 15 handshakes. These results suggest that post-quantum key exchange selection can be treated as a controlled telemetry system parameter, rather than as a fixed deployment choice.

Future work should evaluate the same approach on hardware closer to deployed telemetry systems and with more realistic link traces. Important extensions include packet loss, jitter, gradual bandwidth drift, contact window timing, and embedded processor constraints. These tests would show how well adaptive scheme selection transfers from controlled testbed conditions to operational telemetry links.

REFERENCES

[1] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, pp. 124–134, IEEE, 1994.

[2] National Institute of Standards and Technology, "Module-lattice-based key-encapsulation mechanism standard," Federal Information Processing Standards Publication FIPS 203, National Institute of Standards and Technology, 2024.

[3] N. Bindel, J. Brendel, M. Fischlin, B. Gonçalves, and D. Stebila, "Hybrid key encapsulation mechanisms and authenticated key exchange," in *Post-Quantum Cryptography – PQCrypto 2019*, vol. 11505 of *Lecture Notes in Computer Science*, pp. 206–226, Springer, 2019.

[4] J. W. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM," in *2018 IEEE European Symposium on Security and Privacy*, pp. 353–367, IEEE, 2018.

[5] J. W. Bos, C. Costello, M. Naehrig, and D. Stebila, "Post-quantum key exchange for the TLS protocol from the ring learning with errors problem," in *2015 IEEE Symposium on Security and Privacy*, pp. 553–570, IEEE, 2015.

[6] C. Paquin, D. Stebila, and G. Tamvada, "Benchmarking post-quantum cryptography in TLS," in *Post-Quantum Cryptography – PQCrypto 2020*, vol. 12100 of *Lecture Notes in Computer Science*, pp. 72–91, Springer, 2020.

[7] B. Arnesen, T. Wai, T. Randall, Y. Yang, D. Mitchell, and W. K. Harrison, "Applying post-quantum cryptography based on quasi-cyclic codes to aeronautical mobile telemetry," in *Proceedings of the International Telemetering Conference*, vol. 60, 2025.

[8] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, pp. 212–219, ACM, 1996.

[9] E. Alkim, L. Ducas, T. Pöppelmann, and P. Schwabe, "Post-quantum key exchange—a new hope," in *25th USENIX Security Symposium*, pp. 327–343, USENIX Association, 2016.

[10] E. Rescorla, "The transport layer security (TLS) protocol version 1.3," RFC 8446, Internet Engineering Task Force, 2018.

[11] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, "Post-quantum authentication in TLS 1.3: A performance study," in *Proceedings of the Network and Distributed System Security Symposium*, Internet Society, 2020.

[12] D. Stebila and M. Mosca, "Post-quantum key exchange for the internet and the open quantum safe project," in *Selected Areas in Cryptography – SAC 2016*, vol. 10532 of *Lecture Notes in Computer Science*, pp. 14–37, Springer, 2017.

[13] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Machine Learning*, vol. 47, no. 2–3, pp. 235–256, 2002.

[14] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in *Proceedings of the 19th International Conference on World Wide Web*, pp. 661–670, ACM, 2010.

[15] Y. Abbasi-Yadkori, D. Pál, and C. Szepesvári, "Improved algorithms for linear stochastic bandits," in *Advances in Neural Information Processing Systems*, vol. 24, 2011.

[16] A. Garivier and E. Moulines, "On upper-confidence bound policies for switching bandit problems," in *Algorithmic Learning Theory*, vol. 6925 of *Lecture Notes in Computer Science*, pp. 174–188, Springer, 2011.